Лабораторная работа по криптографии №4
$Жуховицкий\ А.\ Д.$ (М8О-303Б-23)
Этот Jupyter на GitHub

Задача: подобрать эллиптическую кривую над конечным простым полем $\mathbb{Z}_p$ и точку на ней так, чтобы порядок этой точки при полном переборе находился примерно за 10 минут на данном ПК.

В работе используется кривая вида

$$E : y^2 \equiv x^3 + ax + b \pmod{p},$$

где p — простое число, а дискриминант не равен нулю:

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}.$$

Идея решения:

1. По замеренной скорости полного перебора вычислить требуемый диапазон порядка точки.
2. Подбирать простое поле $\mathbb{Z}_p$, параметры кривой a, b и случайные точки P.
3. Быстро оценивать порядок точки с помощью baby-step giant-step, чтобы не ждать 10 минут на каждом кандидате.
4. После нахождения подходящей точки проверить её порядок полным перебором и зафиксировать время.

Импорт библиотек и исходные параметры

`REAL_OPS` — реальная скорость выполнения операции сложения точек, полученная из long-run замера. По ней считается, какой порядок точки даст время полного перебора в интервале 10–15 минут.

```
from math import isqrt
from random import randrange, seed
from time import perf_counter
import platform
import cpuinfo
import os

O = None

# Замеренная скорость полного перебора из long-run.
# 2_162_413_821 сложение за 1725.3949038329883 сек.
REAL_OPS = 2162413821 / 1725.3949038329883
```

```
MIN_MINUTES = 13
MAX_MINUTES = 18

seed(42)
```

Характеристики вычислителя

По ТЗ в отчёте нужно указать характеристики вычислителя. Следующая ячейка выводит основные сведения о машине, на которой запускается ноутбук.

```
def print_computer_info():
    print("ОС:", platform.platform())
    print("Процессор:", cpuinfo.get_cpu_info()["brand_raw"])
    print("Python:", platform.python_version())
    print("Логических CPU:", os.cpu_count())


print_computer_info()
```

```
ОС: macOS-15.7.5-arm64-arm-64bit-Mach-O
Процессор: Apple M4 Pro
Python: 3.14.0
Логических CPU: 14
```

Расчёт требуемого порядка точки

Если полный перебор делает `REAL_OPS` сложений точек в секунду, то за 10–15 минут он выполнит примерно такой диапазон операций:

$$N_{min} = v \cdot 10 \cdot 60, \quad N_{max} = v \cdot 15 \cdot 60.$$

Именно в этот диапазон должен попасть порядок точки $\mathrm{ord}(P)$.

```
min_order = int(REAL_OPS * MIN_MINUTES * 60)
max_order = int(REAL_OPS * MAX_MINUTES * 60)
target_order = (min_order + max_order) // 2

print(f"Реальная скорость: {REAL_OPS:.2f} сложений/сек")
print(f"Нужный порядок точки для 10–15 минут: [{min_order}, {max_order}]")
print(f"Целевой порядок: около {target_order}")
```

```
Реальная скорость: 1253286.32 сложений/сек
Нужный порядок точки для 10–15 минут: [977563325, 1353549220]
Целевой порядок: около 1165556272
```

Арифметика в поле $\mathbb{Z}_p$

Для операций над точками нужна обратная величина по модулю p. В Python её удобно получать через `pow(x, -1, p)`.

```python
def inv_mod(x, p):
    """Обратный элемент к x по модулю p."""
    return pow(x, -1, p)
```

Сложение точек эллиптической кривой

Функция `ec_add` реализует групповую операцию на кривой.

Обрабатываются основные случаи:

- одна из точек является нейтральным элементом `None`;
- точки противоположны, поэтому сумма равна бесконечно удалённой точке;
- сложение разных точек;
- удвоение точки.

```python
def ec_add(P, Q, a, p):
    """Сложение двух точек P и Q на кривой y^2 = x^3 + ax + b mod p."""
    if P is None:
        return Q
    if Q is None:
        return P

    x1, y1 = P
    x2, y2 = Q

    # P + (-P) = 0
    if x1 == x2 and (y1 + y2) % p == 0:
        return None

    if P == Q:
        # Касательная вертикальна
        if y1 % p == 0:
            return None
        lam = ((3 * x1 * x1 + a) * inv_mod(2 * y1, p)) % p
    else:
        lam = ((y2 - y1) * inv_mod(x2 - x1, p)) % p

    x3 = (lam * lam - x1 - x2) % p
    y3 = (lam * (x1 - x3) - y1) % p
    return x3, y3
```

Умножение точки на число

`ec_mul(k, P, a, p)` вычисляет kP методом double-and-add. Это быстрее, чем складывать точку P ровно k раз.

```python
def ec_mul(k, P, a, p):
    """Быстрое умножение точки P на целое k методом double-and-add."""
    R = None
    Q = P
```

```
    while k > 0:
        if k & 1:
            R = ec_add(R, Q, a, p)
        Q = ec_add(Q, Q, a, p)
        k >>= 1
    return R
```

Проверка квадратичного вычета и извлечение корня

Чтобы построить случайную точку кривой, выбираем случайный x и проверяем, является ли $x^3 + ax + b \pmod{p}$ квадратичным вычетом, используя символ Лежандра. Если да, извлекаем квадратный корень алгоритмом Тонелли—Шенкса.

```
def legendre_symbol(n, p):
    """Символ Лежандра: 0, 1 или p-1, где p-1 соответствует -1 mod p."""
    if n % p == 0:
        return 0
    return pow(n, (p - 1) // 2, p)


def tonelli_shanks(n, p):
    """Возвращает один квадратный корень из n по модулю простого p или None."""
    n %= p
    if n == 0:
        return 0
    if p == 2:
        return n
    if legendre_symbol(n, p) != 1:
        return None
    if p % 4 == 3:
        return pow(n, (p + 1) // 4, p)

    q = p - 1
    s = 0
    while q % 2 == 0:
        s += 1
        q //= 2

    z = 2
    while legendre_symbol(z, p) != p - 1:
        z += 1

    m = s
    c = pow(z, q, p)
    t = pow(n, q, p)
    r = pow(n, (q + 1) // 2, p)

    while t != 1:
        i = 1
        t2i = (t * t) % p
        while t2i != 1:
            t2i = (t2i * t2i) % p
            i += 1
```

```
        b = pow(c, 1 << (m - i - 1), p)
        m = i
        c = (b * b) % p
        t = (t * c) % p
        r = (r * b) % p

    return r
```

Поиск простых чисел

Поле должно быть простым: $\mathbb{Z}_p$. Поэтому требуется проверка простоты и функция поиска ближайшего простого числа.

```
def is_probable_prime(n):
    """Вероятностная проверка простоты Миллера–Рабина с фиксированными основаниями."""
    if n < 2:
        return False

    small_primes = (2, 3, 5, 7, 11, 13, 17, 19, 23, 29)
    for q in small_primes:
        if n % q == 0:
            return n == q

    d = n - 1
    s = 0
    while d % 2 == 0:
        s += 1
        d //= 2

    for a in (2, 3, 5, 7, 11, 13):
        if a >= n:
            continue
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False

    return True


def next_prime(n):
    """Находит ближайшее простое число >= n."""
    if n <= 2:
        return 2
    n = n + 1 if n % 2 == 0 else n
    while not is_probable_prime(n):
```

```
        n += 2
    return n
```

Факторизация и делители

Эти функции нужны для уменьшения найденного порядка точки: если $nP = O$, проверяем делители n, чтобы найти минимальный положительный порядок.

```
def factorize(n):
    """Пробная факторизация числа n."""
    factors = {}
    d = 2
    while d * d <= n:
        while n % d == 0:
            factors[d] = factors.get(d, 0) + 1
            n //= d
        d = 3 if d == 2 else d + 2
    if n > 1:
        factors[n] = factors.get(n, 0) + 1
    return factors


def all_divisors(factors):
    """Строит список всех делителей по факторизации."""
    res = [1]
    for p, e in factors.items():
        cur = []
        for d in res:
            mul = 1
            for _ in range(e + 1):
                cur.append(d * mul)
                mul *= p
        res = cur
    return sorted(res)
```

Получение случайной точки на кривой

Функция выбирает случайный x, вычисляет правую часть уравнения кривой и пытается найти y. Если корень существует, точка принадлежит кривой.

```
def random_curve_point(a, b, p, tries=5000):
    """Ищет случайную точку на кривой y^2 = x^3 + ax + b mod p."""
    for _ in range(tries):
        x = randrange(0, p)
        rhs = (x * x * x + a * x + b) % p
        ls = legendre_symbol(rhs, p)
        if ls in (0, 1):
            y = tonelli_shanks(rhs, p)
            if y is not None:
                return (x, y)
    return None
```

Быстрое нахождение порядка точки: baby-step giant-step

Полный перебор порядка точки может занимать 10 минут и более, поэтому для подбора кандидатов используется метод baby-step giant-step.

Для верхней границы используется теорема Хассе:

$$p + 1 - 2\sqrt{p} \leq |E(\mathbb{Z}_p)| \leq p + 1 + 2\sqrt{p}.$$

Поскольку порядок точки делит порядок группы, эта оценка даёт разумную верхнюю границу для поиска.

```
def bsgs_order(P, a, p, upper_bound):
    """Находит порядок точки P методом baby-step giant-step."""
    m = isqrt(upper_bound) + 1

    # Baby steps: jP
    baby = {}
    R = None
    for j in range(m + 1):
        if R not in baby:
            baby[R] = j
        R = ec_add(R, P, a, p)

    # Giant steps: -i*mP
    mP = ec_mul(m, P, a, p)
    neg_mP = None if mP is None else (mP[0], (-mP[1]) % p)

    gamma = None
    candidate = None

    for i in range(1, m + 2):
        gamma = ec_add(gamma, neg_mP, a, p)
        j = baby.get(gamma)
        if j is not None:
            n = i * m + j
            if n > 0 and ec_mul(n, P, a, p) is None:
                candidate = n
                break

    if candidate is None:
        return None

    # Делим кандидат на простые множители, пока свойство nP = O сохраняется.
    fac = factorize(candidate)
    for q, e in fac.items():
        for _ in range(e):
            reduced = candidate // q
            if ec_mul(reduced, P, a, p) is None:
                candidate = reduced
            else:
```

```
                break

    return candidate
```

Подгонка порядка точки

Если случайная точка имеет слишком большой порядок, можно получить точку меньшего порядка: если $\mathrm{ord}(P) = n$ и d — делитель n, то точка

$$Q = \frac{n}{d} P$$

имеет порядок, делящий d. В коде выбирается делитель, который попадает в требуемый диапазон 10–15 минут.

```
def tune_point_order(P, ordP, a, p, min_order, max_order):
    """Пытается получить из P точку Q с порядком в нужном диапазоне."""
    divisors = all_divisors(factorize(ordP))
    good = [d for d in divisors if min_order <= d <= max_order]
    if not good:
        return None, None

    target = min(good, key=lambda d: abs(d - (min_order + max_order) // 2))
    k = ordP // target
    Q = ec_mul(k, P, a, p)
    if Q is None:
        return None, None
    return Q, target
```

Полный перебор порядка точки

Это контрольная функция для финальной проверки. Она последовательно считает $P, 2P, 3P, \ldots$ до тех пор, пока не получит нейтральный элемент O.

Именно этот замер должен попасть в отчёт как экспериментальное подтверждение выполнения условия.

```
def brute_force_order_full(P, a, p):
    """Находит порядок точки полным перебором и возвращает (порядок, время)."""
    Q = None
    n = 0
    start = perf_counter()
    while True:
        Q = ec_add(Q, P, a, p)
        n += 1
        if Q is None:
            return n, perf_counter() - start
```

Основной алгоритм подбора кривой

Алгоритм:

1. Считает диапазон требуемых порядков.
2. Берёт простое p около целевого порядка.
3. Случайно генерирует параметры a, b.
4. Проверяет невырожденность кривой.
5. Генерирует случайные точки.
6. Быстро находит порядок точки через baby-step giant-step.
7. Если порядок подходит — возвращает результат.
8. Если порядок не подходит, пытается получить точку подходящего порядка через делитель порядка исходной точки.

```
def find_lab_curve():
    min_order = int(REAL_OPS * MIN_MINUTES * 60)
    max_order = int(REAL_OPS * MAX_MINUTES * 60)
    target_order = (min_order + max_order) // 2

    print(f"Реальная скорость: {REAL_OPS:.2f} сложений/сек")
    print(f"Нужный порядок точки для 10–15 минут: [{min_order}, {max_order}]")
    print(f"Целевой порядок: около {target_order}")

    p0 = next_prime(target_order)

    for delta in range(0, 400000, 10000):
        for p in {next_prime(p0 + delta), next_prime(max(3, p0 - delta))}:
            hasse_low = p + 1 - 2 * isqrt(p)
            hasse_high = p + 1 + 2 * isqrt(p)

            for _ in range(120):
                a = randrange(0, p)
                b = randrange(1, p)

                # Проверка невырожденности кривой
                if (4 * pow(a, 3, p) + 27 * pow(b, 2, p)) % p == 0:
                    continue

                for _ in range(8):
                    P = random_curve_point(a, b, p)
                    if P is None:
                        continue

                    ordP = bsgs_order(P, a, p, hasse_high)
                    if ordP is None:
                        continue

                    if min_order <= ordP <= max_order:
                        est = ordP / REAL_OPS / 60
                        return {
                            "p": p,
                            "a": a,
                            "b": b,
                            "P": P,
```

```
                            "order": ordP,
                            "est_minutes": est,
                            "hasse_low": hasse_low,
                            "hasse_high": hasse_high,
                        }

                    Q, tuned_order = tune_point_order(P, ordP, a, p, min_order, max_o
                    if Q is not None:
                        real_order = bsgs_order(Q, a, p, hasse_high)
                        if real_order is not None and min_order <= real_order <= max_
                            est = real_order / REAL_OPS / 60
                            return {
                                "p": p,
                                "a": a,
                                "b": b,
                                "P": Q,
                                "order": real_order,
                                "est_minutes": est,
                                "hasse_low": hasse_low,
                                "hasse_high": hasse_high,
                            }

    return None
```

Запуск подбора

Эта ячейка ищет подходящую кривую. Время выполнения зависит от компьютера и случайных параметров. Если результат не найден, можно увеличить диапазон `delta`, количество кривых или количество точек на кривую.

```
search_start = perf_counter()
result = find_lab_curve()
search_time = perf_counter() - search_start

if result is None:
    print("Не удалось найти подходящую кривую. Увеличьте диапазон поиска.")
else:
    print("Найдена подходящая кривая:")
    print(f"p = {result['p']}")
    print(f"a = {result['a']}")
    print(f"b = {result['b']}")
    print(f"P = {result['P']}")
    print(f"ord(P) = {result['order']}")
    print(f"Оценка времени полного перебора = {result['est_minutes']:.2f} минут")
    print(f"Интервал Хассе: [{result['hasse_low']}, {result['hasse_high']}]")
    print(f"Время подбора = {search_time:.2f} сек")
```

```
Реальная скорость: 1253286.32 сложений/сек
Нужный порядок точки для 10–15 минут: [977563325, 1353549220]
Целевой порядок: около 1165556272
Найдена подходящая кривая:
p = 1165556291
a = 239081663
b = 53710185
P = (525901256, 425494765)
ord(P) = 1165523787
Оценка времени полного перебора = 15.50 минут
Интервал Хассе: [1165488012, 1165624572]
Время подбора = 0.05 сек
```

Контрольная проверка полным перебором

```python
if result is None:
    print("Сначала нужно найти подходящую кривую.")
else:
    n, t = brute_force_order_full(result["P"], result["a"], result["p"])
    print("Полный порядок:", n)
    print("Время полного перебора:", t, "сек")
    print("Время в минутах:", t / 60)

    assert n == result["order"], "Порядок из BSGS не совпал с полным перебором"
```

```
Полный порядок: 1165523787
Время полного перебора: 812.0054666670039 сек
Время в минутах: 13.533424444450066
```